



Centro Integrado de Formación Profesional
AVILÉS
Principado de Asturias

UNIDAD 7: INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

DESARROLLO WEB EN ENTORNO CLIENTE

2º CURSO

C.F.G.S. DISEÑO DE APLICACIONES WEB

REGISTRO DE CAMBIOS

Versión	Fecha	Estado	Resumen de cambios
1.0	16/02/2025	Aprobado	Primera versión
1.1	23/02/2025	Aprobado	Eliminado apartado en 7.1
1.2	01/04/2025	Aprobado	Corrección de errores
2.1	17/12/2025	Aprobado	Actualización 2025
2.2	15/04/2025	Aprobado	Corrección de errores

ÍNDICE

ÍNDICE	1
UNIDAD 7: INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS.....	3
7.1 Modelo de gestión de eventos. Tipos. Manejadores de eventos	3
7.1 Closures.....	6
7.2 Eventos de teclado.....	7
7.3 Eventos personalizados	8
7.4 Gestión de eventos en React.....	8
7.2 El formulario. Los objetos del formulario. Propiedades y métodos de un formulario	10
7.2.1 El elemento input.....	12
7.2.2 Formularios en React	15
7.3 Utilización de formularios desde código. Acceso a los miembros del formulario.....	16
Campos de archivo.....	17
7.4 Modificación de apariencia y comportamiento	18
7.5 Validación y envío. Procesamiento de datos en un formulario.....	20
7.6 Rutinas del lenguaje de script para la aplicación a formularios.....	23
7.7 Expresiones regulares.....	25
7.8 Prueba y documentación del código	28
ÍNDICE DE FIGURAS.....	32

BIBLIOGRAFÍA – WEBGRAFÍA	32
--------------------------------	----

UNIDAD 7: INTERACCIÓN CON EL USUARIO: EVENTOS Y FORMULARIOS

7.1 MODELO DE GESTIÓN DE EVENTOS. TIPOS. MANEJADORES DE EVENTOS

Una de las características básicas de JavaScript es la capacidad que tiene para capturar determinadas acciones producidas por el usuario, conocidas habitualmente como eventos. JavaScript detecta determinadas acciones producidas por el usuario, como son pulsar un objeto, enviar un formulario, modificar la información de un campo del formulario, movimiento del ratón sobre un objeto, etc. También detecta algunos eventos no relacionados directamente con acciones de usuario, como la carga o descarga de un documento en el navegador. Cada evento tiene asociado un nombre (**onclick**, **onsubmit**, **onload**, etc.), de forma que desde una página HTML se puede indicar qué acción se desea realizar cuando se genere un determinado evento en un elemento de la página. Por ejemplo, si se dispone de una página en la que se ha definido un formulario, es posible indicar que cuando se pulse el botón "Submit" se ejecute un script que valide los datos del formulario en la máquina del cliente y si todo es correcto que los envíe al servidor web para que este los procese.

En la programación orientada a eventos, las aplicaciones esperan a que se produzcan los eventos. Una vez que se produce un evento, la aplicación responde ejecutando cierto código especialmente preparado. Este tipo de código se denomina "**manejadores de eventos**" (del inglés "**event handlers**") y las funciones externas que se definen para responder a los eventos se suelen denominar "funciones manejadoras".

Anteriormente ya se ha visto cómo manejar eventos y asociarlos a un elemento del DOM mediante **addEventListener**. La concepción clásica del manejo de eventos por HTML consiste en asociar el manejador directamente desde el componente mediante un atributo con el mismo nombre. En el caso del evento **click**, el manejador sería **onclick**.

Un ejemplo de uso podría ser el siguiente:

```

```

La especificación DOM define cuatro grupos de eventos dividiéndolos según su origen:

- Eventos del ratón: click, dblclick, mousedown, mouseup, mouseover, mousemove
- Eventos del teclado: keydown, keypress, keyup
- Eventos HTML: load, unload, abort, error, select, change, submit, reset, resize, etc.

En la siguiente tabla se puede consultar una lista de los eventos que pueden darse en elementos HTML junto con su descripción.

Evento	Descripción
onload	Al terminar la carga de una página
onunload	Cuando se elimina un documento de la ventana
onabort	Cuando el usuario detiene la descarga de un elemento antes de que haya terminado
onerror	Cuando se ha producido un error en JavaScript
onreset	Cuando se pulsa un botón de reset
onclick	Cuando se hace clic sobre un elemento
ondblclick	Cuando se hace doble clic sobre un elemento
onmousedown	Cuando se pulsa con el ratón sobre un elemento
onmouseup	Cuando se levanta el botón del ratón de un elemento
onmouseover	Cuando el ratón entra dentro de un elemento
onmousemove	Cuando el ratón se mueve estando dentro de un elemento
onmouseout	Cuando el ratón sale de la zona de un elemento
onfocus	Cuando se activa el foco en un elemento
onblur	Cuando un elemento deja de estar activo
onkeypress	Cuando una tecla se pulse y levanta sobre un elemento
onkeydown	Cuando una tecla es pulsada sobre un elemento
onkeyup	Cuando una tecla es levantada de un elemento
onsubmit	Cuando se envía la información de un formulario
onreset	Cuando se limpia el contenido de un formulario
onselect	Cuando el usuario selecciona texto de una entrada
onchange	Cuando el elemento cambia su valor

En los eventos de JavaScript, se puede utilizar la palabra reservada **this** para referirse al elemento HTML sobre el que se está ejecutando el evento. Esta técnica es útil para ejemplos como el siguiente, en el que se modifican las propiedades del elemento que provoca el evento. En el siguiente código, se muestra un borde destacado en el elemento <div> cuando el usuario pasa el ratón por encima. Cuando el ratón sale del <div>, se vuelve a mostrar el borde original:

```
<div
  id="elemento"
  style="padding: 0.2em;
    width: 150px;
    height: 60px;
    border: thin solid silver;"
  onmouseover="document.getElementById('elemento').style.borderColor='black'"
  onmouseout="document.getElementById('elemento').style.borderColor='silver'">
  Sección de contenidos...
</div>
```

El evento **onmouseover** se activa cuando el ratón pasa por encima del elemento, en este caso el **<div>**. Para cambiar el color del borde, se utiliza la propiedad **border-color** de CSS. Por lo tanto, en primer lugar se obtiene la referencia del elemento mediante **document.getElementById('elemento')**. A continuación, se utiliza la propiedad **style** para acceder a las propiedades CSS y se modifica el nombre de **border-color** por el de **borderColor**. Para volver al color original cuando el ratón sale del elemento, se realiza la misma operación sobre el evento **onmouseout**.

El uso de la variable **this** puede simplificar todos estos pasos, ya que, dentro de un manejador de eventos, la variable **this** equivale al elemento que ha provocado el evento. Así, la variable **this** es igual al **<div>** de la página y por tanto **this.style.borderColor** permite cambiar de forma directa el color del borde del **<div>**:

```
<div
  style="padding: 0.2em;
    width: 150px;
    height: 60px;
    border: thin solid silver;"
  onmouseover="this.style.borderColor='black'"
  onmouseout="this.style.borderColor='silver'">
  Sección de contenidos...
</div>
```

Haciendo uso de la variable **this**, el código es mucho más sencillo de escribir, leer y mantener.

La definición de manejadores de eventos en los atributos HTML es un método sencillo, pero poco aconsejable para tratar con los eventos en JavaScript. El principal inconveniente es que se complica en exceso en cuanto se añaden algunas pocas instrucciones, por lo que solamente es recomendable para los casos más sencillos.

Cuando el código de la función manejadora es más complejo, como por ejemplo la validación de un formulario, es aconsejable agrupar todo el código JavaScript en una función externa que se invoca desde el código HTML cuando se produce el evento.

De esta forma, el siguiente ejemplo:

```
<button onclick="alert('Gracias por hacer clic');">Pulsa y verás</button>
```

Se puede transformar en:

```
<script>
  const muestraMensaje = () => {
    alert("Gracias por hacer clic");
  };
</script>

<button onclick="muestraMensaje()">Pulsa y verás</button>
```

7.1 CLOSURES

Una clausura o closure se puede definir como una expresión que se evalúa en tiempo de ejecución. A continuación, se verá un ejemplo de uso.

Se define un objeto **MsgBox** mediante notación JSON que tiene un atributo **msg** con el mensaje a mostrar y dos métodos: **setMsg** y **showMsg** que establecen y muestran el mensaje respectivamente. El código es el siguiente:

```
let MsgBox = {  
  msg : null,  
  setMsg: function (msg)  
  {  
    this.msg = msg;  
  },  
  showMsg: function()  
  {  
    var self = this;  
    function delayedShowMsg(){  
      alert(self.msg);  
    }  
    setTimeout(delayedShowMsg, 1000);  
  }  
};
```

El método **showMsg** va a tener una particularidad: el mensaje se va a mostrar con un cierto retraso de un segundo después de que se ejecute el método. **self** es una referencia al propio objeto. En el método **delayedShowMsg** lo que se desea es mostrar el mensaje de alerta, pero no se sabe a priori dónde está ese mensaje. Esta función se ejecuta fuera del flujo normal de ejecución del programa y no se conoce qué variables ve. Hay que preguntarse entonces si estar dentro del código de **showMsg** implica que ve las variables de dicho método y las propiedades del objeto que lo contiene.

Esta expresión es un **closure**: cuando se ejecuta **delayedShowMsg** la variable **self** contiene una referencia válida al objeto **MsgBox** y a partir de ahí resulta fácil llegar a la propiedad **msg**. Observar que **self** es visible a pesar de que **delayedShowMsg** se ejecuta fuera de la línea estándar de ejecución y que **self** guarda el valor que tenía justo antes de definirse la función.

En este tipo de expresiones, uno de los errores más frecuentes consiste en el uso de la palabra reservada "**this**". Por ejemplo, de esta forma:

```
function delayedShowMsg(){  
  alert(this.msg);  
}
```

Esto no funciona porque en el contexto de ejecución **this** no tiene sentido, ya que se hace referencia a sí misma y no al objeto. El truco es usar esa variable **self** para referenciar al objeto aprovechando las ventajas de los **closure** de JavaScript.

Para probarlo, no hay más que asignar el mensaje y ejecutar el método que lo muestra.

```
MsgBox.setMsg("Hola, mundo con closures");  
MsgBox.showMsg();
```

7.2 EVENTOS DE TECLADO

Cuando se trabaja con el teclado, se pueden usar los eventos **keydown** y/o **keyup**. En ambos casos, hay propiedades como **key**, que representa el carácter pulsado. Con estos eventos es posible desplazarse a zonas de la página, desplegar un menú determinado e incluso realizar pequeños juegos que respondan a la pulsación de las teclas.

```
<head>  
  <script>  
    const mostrarTecla = (e) => {  
      const tecla = e.key;  
  
      // Se filtran teclas especiales  
      if (tecla.length === 1 || tecla === " ")  
        document.getElementById("zonadeteclas").textContent += tecla;  
    };  
  </script>  
</head>  
<body onkeydown="mostrarTecla(event);">  
  Prueba de teclas: <span id="zonadeteclas"></span>  
</body>
```

Cada vez que se pulse una tecla, se mostrará en la zona en la que se ha ubicado la etiqueta ****. Si se quiere comprobar si se ha pulsado alguna tecla especial, también se puede usar la propiedad **key**. Por último, las propiedades **altKey**, **ctrlKey** y **shiftKey** almacenan un valor booleano que indica si alguna de esas teclas estaba pulsada al producirse el evento del teclado.

```
if (e.altKey) alert("Has pulsado alt");
```

Si lo que se quiere es evaluar la combinación de una tecla especial con otra alfanumérica, el código sería así (para detectar Ctrl + C):

```
if (e.ctrlKey && e.key === "c") { console.log("Ctrl + C presionado"); }
```


7.3 EVENTOS PERSONALIZADOS

Se pueden crear eventos personalizados de la siguiente manera:

```
<button id="elementoPrueba">Elemento de prueba</div>
<script>
  const evento = new Event("personalizado");
  const elementoPrueba = document.getElementById("elementoPrueba");
  elementoPrueba.addEventListener("personalizado", (e) => {
    alert("Has lanzado el evento personalizado");}, false);
  // Se dispara el evento personalizado al hacer clic sobre el botón:
  elementoPrueba.addEventListener('click', () => {
    elementoPrueba.dispatchEvent(evento);});
</script>
```

También es posible crear eventos propios, incluso enviando datos por ellos. Basta con hacer lo siguiente:

```
const elementoPrueba = document.getElementById("elementoPrueba");
const evento = new CustomEvent("personalizado2", {
  detail:{id: 1}
});
elementoPrueba.addEventListener(
  "personalizado2",
  (e) => {
    console.info("Evento disparado:", e);
    console.info("Datos del evento:", e.detail)
  }
);

elementoPrueba.addEventListener('click', () => {
  elementoPrueba.dispatchEvent(evento);
});
```

7.4 GESTIÓN DE EVENTOS EN REACT

El manejo de eventos en React, como ya se ha visto con anterioridad, no es exactamente igual al de **Vanilla JavaScript**. Ello es debido a que usa un sistema conocido como eventos sintéticos (**Synthetic Events**). Este sistema es en realidad una herramienta muy potente para manejar las interacciones del usuario en distintos navegadores de forma eficiente y consistente.

Los eventos sintéticos son un envoltorio de los eventos nativos del navegador que permite su uso sin tener en cuenta en cuál de ellos se utiliza. Esto es porque ofrecen una API consistente que asegura que los eventos funcionen exactamente del mismo modo en cualquier navegador. Un objeto de evento sintético replica de forma bastante exacta la estructura de los eventos nativos DOM de forma estandarizada.

Las características más importantes de este tipo de eventos son:

- **Compatibilidad entre navegadores.** Proporcionan una interfaz consistente entre todos los navegadores eliminando la necesidad de estar pendientes de las particularidades de cada navegador
- **Agrupación de eventos.** **React** optimiza el rendimiento mediante la agrupación de eventos, lo que significa que los objetos de eventos sintéticos se reutilizan para lograr una mayor eficiencia. Una vez que se maneja un evento, el objeto de evento se borra para su reutilización. Sin embargo, si necesita acceder al evento de forma asíncrona, se puede llamar a **event.persist()** para evitar que **React** lo borre.
- **Propagación de eventos.** Como los eventos nativos DOM, los sintéticos siguen el modelo de propagación de eventos incluyendo la fase de captura y de burbuja. Esto permite manejar la delegación de eventos y los manejadores tanto en el elemento objetivo como en sus padres.
- **Prevención del comportamiento por defecto.** Se puede prevenir el comportamiento por defecto del navegador usando el método **event.preventDefault()** el cual funciona de forma similar a como lo hace **Vanilla JavaScript**.

Ejemplos sencillos de uso:

1. Escribiendo manejadores de evento. En este ejemplo se puede ver un manejador sencillo que responde al clic de un botón.

```
const manejaClick = (event) => {  
  console.log('Botón pulsado:', event);  
}  
<button onClick={manejaClick}>Púlsame</button>
```

2. Manejando eventos de formulario. En este caso se maneja la entrada del usuario en el control.

```
const manejaCambioInput = (event) => {  
  console.log('Valor del input:', event.target.value);  
}  
<input type="text" onChange={manejaCambioInput} />
```

3. Previniendo el comportamiento por defecto. Al igual que en **Vanilla JavaScript**, se evita que se envíe el formulario ya que es su comportamiento por defecto.

```
const manejaSubmit = (event) => {  
  event.preventDefault();  
  console.log('Formulario enviado');  
}  
<form onSubmit={manejaSubmit}>  
  <button type="submit">Enviar</button>  
</form>
```

7.2 EL FORMULARIO. LOS OBJETOS DEL FORMULARIO. PROPIEDADES Y MÉTODOS DE UN FORMULARIO

Un formulario web sirve para enviar, tratar y recuperar datos que son enviados y recibidos entre un cliente y un servidor web.

Cada elemento del formulario almacena un tipo de dato o acciona una de sus funcionalidades.

Los formularios disponen de una arquitectura y en este contexto están enmarcados en el lenguaje HTML.

Los formularios se definen con etiquetas **<form>**. Para que sea funcional, la etiqueta necesita inicializar dos atributos:

- **action**. Contiene la URL donde se redirigen los datos del formulario.
- **Method**. Indica el método por el cual el formulario envía los datos. Puede ser POST o GET.

Un ejemplo de formulario sencillo:

```
<html>
  <head>
    <title>Ejemplo de formulario</title>
  </head>
  <body>
    <h3>Formulario</h3>
    <form action="www.Web.es/formulario.php" method="post">
      ...
    </form>
  </body>
</html>
```

Las propiedades del objeto form son:

Propiedad	Descripción
action	Especifica el agente procesador del envío. Debe ser una página PHP en el servidor o una dirección de correo electrónico. <code>action = "mailto:email@serv.dom?subject=asunto"</code>
enctype	Indica la forma en la que viajará la información. application/x-www-form-urlencoded Es el valor por defecto. Los datos del FORM se codifican como pares de nombre/valor. text/plain Es el formato más habitual. Este formato es el que debe usarse con get . multipart/form-data Envía ficheros empaquetados para enviarlos en una sección separada.
method	Método HTTP usado para enviar el formulario. Su valor puede ser get o post . get Envía el conjunto de datos del FORM (en una cadena visible) en la URL (después de la URL y ?). Si se envía más de un dato, irán separados por & . post Envía el conjunto de datos del FORM en el cuerpo de la petición, de forma invisible.
name	Cadena que contiene el valor del atributo name del tag form
target	Indica la ventana o iframe de destino en la que se visualizará la respuesta de tratamiento del formulario, en otras palabras, nombre del objeto window que responde al submit del formulario. El valor es el mismo que el utilizado en HTML (blank , self , top , nombre_marco , etc..)

En cuanto a los métodos:

Método	Descripción
reset()	Resetea el formulario: tiene el mismo efecto que si pulsáramos un botón de tipo RESET dispuesto en el formulario.
submit()	Envía el formulario: tiene el mismo efecto que si pulsáramos un botón de tipo SUBMIT dispuesto en el formulario.

Eventos más utilizados:

Evento	Significado
onsubmit()	Se produce cuando el usuario envía el formulario. Botón submit.
onreset()	Se produce al resetear el valor de sus componentes . Botón reset.
onmousemove	Cuando se mueve el ratón.
onmousedown	Se produce cuando el usuario pulsa sobre un botón del ratón.
onmouseup	Cuando suelta un botón del ratón.

7.2.1 EL ELEMENTO INPUT

Uno de los elementos más utilizados en un formulario es el etiquetado como input. Algunos de sus atributos son:

- **type**. Indica el tipo de elemento a definir. De él dependen el resto de parámetros. Valores posibles que acepta el **type**:
 - **text**: cuadro de texto. Se puede usar un elemento **textarea** si se necesita más espacio
 - **password**: cuadro de contraseña, los caracteres aparecen ocultos tras asteriscos.
 - **checkbox**: casilla de verificación. Es mejor utilizar la etiqueta `<select multiple>`
 - **radio**: opción de entre dos o más. Su alternativa más moderna es `<select>`
 - **submit**: botón de envío del formulario. Mejor sustituirlo por `<button type="submit">`
 - **reset**: botón de vaciado de campos. Alternativa moderna: `<button type="reset">`
 - **file**: botón para examinar ficheros.
 - **hidden**: campo oculto para el usuario, ya que no lo visualiza en el formulario
 - **image**: botón de imagen en el formulario.
 - **button**: botón del formulario. La alternativa moderna es **button** ya que, además, ofrece más control sobre estilos y eventos.
- **name**. El atributo **name** asigna un nombre al elemento. Si no se le asigna nombre a un elemento, el servidor no podrá identificarlo de la forma estándar y por tanto no podrá tener acceso al elemento, aunque sí en caso de usarse otras vías.
- **value**. El atributo **value** inicializa el valor del elemento. Los valores dependerán del tipo de dato, en ocasiones los posibles valores a tomar serán verdadero o falso.
- **size**. Este atributo asigna el tamaño inicial del elemento. El tamaño se indica en píxeles. En los campos **text** y **password** hace referencia al número de caracteres.

- **maxlength**. Este atributo indica el número máximo de caracteres que pueden contener los elementos **text** y **password**. Es conveniente saber que el tamaño de los campos text y password es independiente del número de caracteres que acepta el campo.
- **checked**. Este atributo es exclusivo de los elementos **checkbox** y **radio**. En él se define qué opción por defecto se quiere seleccionar.
- **disabled**. Este atributo hace que el elemento aparezca deshabilitado. En este caso el dato no se envía al servidor.
- **readonly**. Este atributo sirve para bloquear el contenido del control, por tanto, el valor del elemento no se podrá modificar.
- **src**. Este atributo es exclusivo para asignar una URL a una imagen que ha sido establecida como botón del formulario.
- **alt**. El atributo **alt** incluye una pequeña descripción del elemento. Habitualmente y si no se ha desactivado cuando se posiciona el ratón encima del elemento (sin pulsar ningún botón), se puede visualizar la descripción de este.

Un ejemplo de formulario completo puede ser el siguiente:

```
<form action="pagina.php" method="post" enctype="multipart/form-data">
  <label for="nombre">Nombre:</label>
  <input type="text" id="nombre" name="nombre" maxlength="30" required />
  <label for="apellidos">Apellidos:</label>
  <input type="text" id="apellidos" name="ape" maxlength="80" required /><br />
  <label for="dni">DNI:</label>
  <input type="text" id="dni" name="dni" maxlength="9" pattern="\d{8}[A-Za-z]"
    required />
  <label for="sexo">Sexo:</label>
  <select id="sexo" name="sexo">
    <option value="hombre" selected>Hombre</option>
    <option value="mujer">Mujer</option>
    <option value="otro">Otro</option>
    <option value="no_especificar">Prefiero no decirlo</option>
  </select><br />
  <label for="foto">Incluir mi foto:</label>
  <input type="file" id="foto" name="foto" accept="image/*" />
  <label for="publicidad">¿Desea recibir publicidad?</label>
  <select id="publicidad" name="publ">
    <option value="si" selected>Sí</option>
    <option value="no">No</option>
  </select><br />
  <button type="submit">Guardar cambios</button>
  <button type="reset">Borrar los datos introducidos</button>
</form>
```

Formulario que tendría un aspecto similar al siguiente:

Nombre: Apellidos:
DNI: Sexo: 
Incluir mi foto: Ningún archivo seleccionado ¿Desea recibir publicidad? 

1. Ejemplo de formulario simple

Para acceder a un formulario también se puede usar la forma correspondiente a un array asociativo:

```
document.forms["miForm"]
```

Obviamente, también se podría por índice, pero no es recomendable dado que podría introducirse algún otro formulario y cambiar el índice volviendo a la aplicación inconsistente.

7.2.2 FORMULARIOS EN REACT

Básicamente, la principal diferencia del uso de formularios en React respecto al JavaScript convencional o **Vanilla JavaScript** es que en este se manipulan directamente los valores de los campos mediante el DOM mientras que **React** se enfoca al estado (con **useState** como ya se ha visto) y en los manejadores de eventos.

Aunque se pueden utilizar otros enfoques, el más utilizado y conveniente en React es el de **formularios controlados**. Este enfoque implica que los valores de inputs y otros elementos de interacción del usuario estén sincronizados con el estado o **state** del componente. Si se quiere modificar un campo del formulario, basta con modificar el estado y viceversa.

En este código puede verse un ejemplo muy sencillo de formulario controlado mediante estados:

```
import { useState } from "react";
const FormControlado = () => {
  const [nombre, setNombre] = useState("");
  const [email, setEmail] = useState("");
  const cambioEntrada = (event) => {
    if (event.target.type === "text") setNombre(event.target.value);
    else if (event.target.type === "email") setEmail(event.target.value);
  };
  const envioForm = (event) => { event.preventDefault();
    alert(`Se ha enviado el formulario con el nombre ${nombre} y correo electrónico
    ${email}`);
  };
  return (
    <form onSubmit={envioForm}>
      <label>Nombre:
        <input type="text" value={nombre} onChange={cambioEntrada} />
      </label>
      <label>Correo electrónico:
        <input type="email" value={email} onChange={cambioEntrada} />
      </label>
      <button type="submit">Enviar datos</button>
    </form>
  );
};
export default FormControlado;
```

La alternativa menos recomendable a este enfoque es usar referencias (**useRef**) y acceder a los valores del virtual DOM. El problema es que en este caso no se aprovecha la naturaleza reactiva de la biblioteca.

7.3 UTILIZACIÓN DE FORMULARIOS DESDE CÓDIGO. ACCESO A LOS MIEMBROS DEL FORMULARIO

Cuando un campo está contenido en un elemento **form**, su elemento DOM tendrá una propiedad con el mismo nombre que se vincula al elemento DOM del formulario. Además, **form** dispone de una propiedad llamada **elements** que contiene una colección similar a una matriz con sus campos.

Como ya se mencionó, el atributo **name** de un campo de formulario determina la manera en que se identificará su valor cuando se envíe. También se puede usar como nombre de propiedad al acceder al **elements**. Un ejemplo:

```
<form action="ejemplo/submit.html">
  Nombre: <input type="text" name="nombre" /><br />
  Contraseña: <input type="password" name="pass" /><br />
  <button type="submit">Entrar</button>
</form>
<script>
  let formulario = document.querySelector('form');
  console.log(formulario.elements[1].type);
  console.log(formulario.elements.pass.type);
  // Ambas líneas de código muestran password
  console.log(formulario.elements.nombre.form == formulario);
  // Muestra true
</script>
```

Enviar un formulario implica que el navegador aparece en la página indicada en el atributo **action**, pero antes de que eso suceda, se activa un evento “**submit**” al que se le puede asociar un manejador que, además, evite que se lance el comportamiento predeterminado llamando a **preventDefault** en el objeto de evento.

```
<form action="ejemplo/submit.html">
  Valor: <input type="text" name="valor" /><br />
  <button type="submit">Guardar</button>
</form>
<script>
  let formulario = document.querySelector('form');
  formulario.addEventListener('submit', (e) => {
    console.log("Guardando valor", formulario.elements.valor.value);
    event.preventDefault();
  });
</script>
```

CAMPOS DE ARCHIVO

Complementando a lo que se vio en la unidad 6, en este apartado se profundizará un poco más sobre el tipo de campo file. Como ya se vio, este tipo de campo se diseñó inicialmente como una manera de cargar ficheros desde el equipo del usuario a través de un formulario. En los navegadores modernos también proporcionan una forma de leer dichos archivos desde programas JavaScript actuando como una especie de guardián. El script no puede simplemente leer archivos privados de un ordenador, pero si se selecciona un archivo en dicho campo, el navegador interpreta que esa acción implica que el script puede leer dicho archivo.

La propiedad **files** de este tipo de elemento es un objeto similar a un array (sin serlo) que contiene los archivos elegidos en el campo. Inicialmente está vacío y la razón por la que existe es debida a que este tipo de componente admite el atributo **multiple** lo que haría que se pudieran seleccionar varios archivos a la vez.

Como se vio anteriormente, los objetos de esta propiedad **files** tienen atributos como **name**, **size**, **type**, etc. Lo que no tiene es una propiedad para el contenido del archivo. Llegar a esto es algo más complicado. En primer lugar, hay que partir de que esta operación lleva un tiempo, por lo que la interfaz debe ser asíncrona para no congelar la página.

```
<input type="file" multiple />
<script>
  let input = document.querySelector("input");
  input.addEventListener("change", () => {
    Array.from(input.files).forEach((f) => {
      let reader = new FileReader();
      reader.addEventListener("load", () => {
        console.log(
          "Fichero",
          f.name,
          "comienza por",
          reader.result.slice(0, 20)
        );
      });
      reader.readAsText(f);
    });
  });
</script>
```

La lectura de un archivo se hace creando un objeto **FileReader**, registrando un manipulador de eventos "load" para él y llamando a su método **readAsText** para darle el archivo a leer. Una vez que acaba la carga, la propiedad **result** del lector alberga el contenido del archivo.

7.4 MODIFICACIÓN DE APARIENCIA Y COMPORTAMIENTO

A través de hojas de estilo es posible mejorar notablemente el aspecto de los formularios. La visualización de un formulario puede depender de ciertas condiciones que vaya introduciendo el usuario. Por defecto los formularios tienen unos estilos asignados que tienen unos colores y unos bordes determinados. A continuación, se verán algunos ejemplos muy simples de aplicación de estilos a controles de un formulario.

En el siguiente ejemplo se modifica el color y borde de un botón:

```
<head>
  <style>
    .azul {
      color: red;
      border-bottom: 4px solid blue;
    }
  </style>
</head>
```

Para aplicar el estilo al botón se utiliza el procedimiento de asignación mediante una clase:

```
<button class="azul">Botón azul</button>
```

Botón normal

Botón azul

2. Comparación entre botón sin estilo aplicado y otro con texto rojo y borde inferior azul

Si se quiere suavizar el aspecto de un cuadro de texto:

```
<style>
  .col {
    background-color: #ffff00;
  }
  .color {
    padding: 0.5em 1em;
    background-color: #ffff00;
  }
</style>
```

A continuación, se aplica cada uno de los estilos a un cuadro de texto:

```
<input class="col" type="text" value="texto" /><br /><br />
<input class="color" type="text" value="texto" />
```

En el primer componente simplemente se le aplica un color de fondo amarillo. Al segundo, se le añade un padding vertical de 0.5em y uno de 1em horizontal.

Ambos componentes quedarían así:



Two text input fields, one above the other, both labeled 'texto'.

3. Cuadros de texto sin y con padding.

Otro ejemplo consiste en organizar los controles de un formulario mediante un elemento **fieldset**:

```
<html>
  <head><style>
    div {
      margin: 0.4em 0;
    }
    div label {
      width: 25%;
      float: left;
    }</style>
  </head>
  <body>
    <fieldset>
      <legend>Formulario</legend>
      <div>
        <label for="nombre">Nombre</label>
        <input type="text" id="nombre" />
      </div>
      <div>
        <label for="apellidos">Apellidos</label>
        <input type="text" id="apellidos" size="35" />
      </div>
      <button class="btn" type="submit">Dar de alta</button>
    </fieldset>
  </body>
</html>
```

El formulario quedaría así:



A form titled 'Formulario' with two text input fields labeled 'Nombre' and 'Apellidos', and a 'Dar de alta' button.

4. Formulario básico organizado

Los formularios tienen unas acciones predeterminadas por defecto. Sin embargo, es posible darle otro comportamiento a uno de sus elementos. Por ejemplo, se podría querer que los datos se envíen a URLs diferentes en base a un dato que introduzca el usuario:

```
const enviar = (form) => {
  form.action = form.alt.a.checked ? "paginas/alta.html" : "paginas/baja.html";
  form.submit()
}
```

7.5 VALIDACIÓN Y ENVÍO. PROCESAMIENTO DE DATOS EN UN FORMULARIO.

Anteriormente se ha visto que es posible acceder a cualquier elemento HTML mediante selectores, lo que podría permitir crear funciones de validación del contenido de cualquier control que permita que el usuario introduzca información. HTML desde su versión 5 introduce elementos que permiten hacer una validación automática en sus controles. Por ejemplo, si se accede a la referencia del control `<input>` en la sección de atributos, existen ejemplos como **max**, que limita el valor máximo de una entrada numérica. Otros posibles atributos son **maxlength**, que determina el máximo tamaño de la cadena a introducir. Además, en la sección [HTML DOM reference](#), se pueden consultar todos los posibles tipos de input, siendo el más clásico el que permite introducir texto, pero como puede verse, se encuentran tipos personalizados para hora, fecha, color, etc. lo que permite limitar el tipo de datos introducido. Algunos ejemplos de uso de estos controles se pueden ver a continuación.

El siguiente código, muestra un formulario con un cuadro de texto con una longitud máxima de 15.

```
<form>
  <table>
    <tr>
      <td>Nombre: </td>
      <td>
        <input type="text" name="nombre" id="nombre" maxlength="15" required>
      </td>
    </tr>
  </table>
</form>
```

Ahora se puede añadir un control para recoger la edad y limitarla a que esté entre 18 y 110 años.

```
<tr>
  <td>Edad: </td>
  <td>
    <input type="number" name="edad" id="edad" min="18" max="110" required>
  </td>
</tr>
```

Todas estas validaciones se comprueban al ir a hacer el **submit** del formulario, pero en un momento dado puede interesar personalizar esta validación.

En un fichero **validacion.js** se incluirá la lógica de validación; dentro de él, se incluye una función en la que se asocia el manejador de evento para el botón.

```
const inicializar = () => document.getElementById('enviar').addEventListener('click',
validar, false);
```

Ahora se crearán los distintos métodos de validación de cada campo. Antes de ello, hay que incluir las siguientes líneas en el encabezado:

```
<script src="./validacion.js"></script>
<style>
    .error{
        border: solid 2px #FFFF00;
    }
</style>
```

El estilo es el que determinará cómo quedará un control cuando esté infringiendo una regla de validación. A continuación de la tabla, se incluirá la línea donde irá el mensaje de error y los botones de envío y de reinicio.

```
<p id="mensajeError"></p>
<p>
    <input type="submit" value="Enviar" id="enviar"></input>
    <input type="reset" value="Borrar" id="borrar"></input>
</p>
```

En el archivo JavaScript se crea un método flexible para la validación de un elemento dado por su nombre. La clave es la función **checkValidity** la cual comprueba si el control cumple con las reglas de validación proporcionadas:

```
const validaElemento = (nombre) => {
    let elemento = document.getElementById(nombre);
    if (!elemento.checkValidity()){
        error(elemento);
        return false;
    }
    return true;
};
```

El método error solo hace que ubicar el mensaje en el párrafo destinado a mostrarlo y le aplica la clase error. Se crea también otra función que es capaz de borrar el error y que no hace otra cosa que eliminar el estilo a los elementos del formulario.

```
const error = (elemento) => {
    document.getElementById('mensajeError').innerHTML = elemento.validationMessage;
    elemento.className = "error";
    elemento.focus();
};

const borrarError = () => {
    let formulario = document.forms[0];
    formulario.elements.forEach((e) => e.className="");
};
```

Finalmente, el método validar, el cual se llama antes del envío, comprueba todas las validaciones y, en caso de no cumplirlas, no hace el **submit** (**e.preventDefault()**).

```
const validar = (e) => {  
  if (validaElemento("nombre") && validaElemento("edad") && confirm("Pulsa enviar  
si quieres mandar el formulario"))  
    return true;  
  else{  
    e.preventDefault();  
    return false;  
  }  
};
```

Hasta aquí se ha visto cómo validar con los elementos que proporciona HTML5 y sus mensajes estándar, pero es bastante probable que se requiera modificar estos mensajes por unos más personalizados. Primeramente, se va a cambiar la función error para que pueda admitir un mensaje opcionalmente y, en ese caso, mostrarlo en el apartado correspondiente.

```
const error = (elemento, mensaje) => {  
  document.getElementById('mensajeError').innerHTML = mensaje === "" ?  
elemento.validationMessage : mensaje;  
  elemento.className = "error";  
  elemento.focus();  
};
```

Se evalúa ahora la validez con la condición de “**valueMissing**” para darle mayor personalización. Se podría combinar esta validación con otras y precisamente este es el objetivo:

```
const validaElemento = (nombre) => {  
  let elemento = document.getElementById(nombre);  
  if (!elemento.checkValidity()){  
    if (elemento.validity.valueMissing)  
      error(elemento, "Debe introducir un nombre")  
    return false;  
  }  
  return true;  
};  
document.addEventListener("DOMContentLoaded", inicializar);
```

Algunas de las bibliotecas más populares para la validación de formularios Javascript son:

- joi <https://joi.dev/>
- Validator.js <https://github.com/validatorjs/validator.js>
- Validate.js <http://rickharrison.github.io/validate.js/>

7.6 RUTINAS DEL LENGUAJE DE SCRIPT PARA LA APLICACIÓN A FORMULARIOS

Las rutinas de script son fragmentos de código reutilizables que permiten realizar tareas comunes relacionadas con formularios, como validación, procesamiento de datos, manipulación de campos o interacción con el usuario. Todas estas operaciones ya han sido vistas tanto en esta unidad como en anteriores. A continuación, se indican las operaciones más comunes realizadas con estas rutinas y los elementos de JavaScript para llevarlas a cabo:

1. Validación de campos de un formulario. En JavaScript este tipo de rutinas se basa en manipular directamente el DOM con métodos selectores como **querySelector** y **addEventListener** para la gestión de eventos. En React se pueden usar variables de estado o referencias directas a elementos del Virtual DOM (menos recomendable).
2. Procesamiento de datos. En este código se introduce la clase **FormData**

```
<html>
  <head><script>
    const procesarDatos = (datos) => {
      return {
        nombreusuario: datos.nombre.trim().toUpperCase(),
        email: datos.email.toLowerCase(),
      };
    };
    document.addEventListener("DOMContentLoaded", () => {
      const formulario = document.querySelector("#infoUsuario");
      formulario.addEventListener("submit", (e) => {
        e.preventDefault();
        const formData = new FormData(formulario);
        const datosProcesados = procesarDatos({
          nombre: formData.get("nombre"),
          email: formData.get("email"),
        });
        console.log("Datos procesados:", datosProcesados);
        alert("Datos enviados correctamente.");
      });
    });
  </script></head>
  <body><form id="infoUsuario">
    <p>
      <label for="nombre">Introduce tu nombre de usuario:</label>
      <input type="text" id="nombre" name="nombre" value="Juan Nadie" />
    </p>
    <p>
      <label for="email">Introduce tu correo electrónico: </label>
      <input type="email" id="email" name="email" required />
    </p>
    <button type="submit">Enviar</button>
  </form></body>
</html>
```


FormData es una clase nativa en JavaScript muy útil para obtener datos de un formulario. En este sentido, funciona como una especie de controlador de dicho elemento. Para ayudar a controlar el comportamiento y los datos de un formulario, la clase **FormData** en JavaScript recibe el nodo HTML, que contiene este componente **form**. Sobre este nodo, la clase permite ejecutar diversos métodos. En el ejemplo, se puede ver que se instancia un objeto de ese tipo proporcionando el formulario obtenido del DOM y mediante el método **get** pasando como parámetro el nombre del elemento de entrada (**name**), se obtiene su valor.

¿Se puede usar **FormData** en React? Aunque se pueda, no se recomienda ya que requiere un acceso al DOM y no es lo recomendable usando **React**. Lo que sí se puede es usar una variable de estado que en realidad sea un objeto al que se le puede llamar `formData`:

```
const [formData, setFormData] = useState({ nombre: "", email: "" });
```

A partir de este momento, se puede usar **formData.nombre** o **formData.email** en el atributo `value`:

```
<input type="text" name="nombre" value={formData.nombre} />
```

7.7 EXPRESIONES REGULARES

Las expresiones regulares describen un conjunto de elementos que siguen un patrón; de hecho, es representan un lenguaje en sí mismas. Un ejemplo podría ser todas las palabras que comienzan por la letra “a” minúscula. JavaScript implementa el lenguaje de expresiones regulares y facilita las comprobaciones de ciertos datos que deben seguir una estructura concreta.

A continuación, se enumeran algunos de los caracteres especiales propios del lenguaje de expresiones regulares:

^ Principio de entrada o línea. Este carácter indica que las cadenas deberán comenzar por el siguiente carácter. Si este fuera una “a” minúscula, la expresión regular sería, ^a.

\$ Fin de entrada o línea. Indica que la cadena debe terminar por el elemento precedido al dólar. Por ejemplo, a\$

*** El carácter anterior 0 o más veces.** El asterisco indica que el carácter anterior se puede repetir en la cadena 0 o más veces.

+ El carácter anterior 1 o más veces. El símbolo más indica que el carácter anterior se puede repetir en la cadena una o más veces.

? El carácter anterior una vez como máximo. El símbolo interrogación indica que el carácter anterior se puede repetir en la cadena cero o una vez.

. **Cualquier carácter individual.** El símbolo punto indica que puede haber cualquier carácter individual salvo el de salto de línea.

x|y “x” ó “y”:La barra vertical indica que puede ser el carácter “x” o el “y”.

{n} n veces el carácter anterior. El carácter anterior a las llaves tiene que aparecer exactamente n veces.

{n,m} Entre n y m veces el carácter anterior. El carácter anterior a las llaves tiene que aparecer como mínimo n y como máximo m veces.

[abc] Cualquier carácter de los corchetes. En la cadena puede aparecer cualquier carácter que este incluido en los corchetes.

[^abc] Un carácter que no esté en los corchetes. En la cadena pueden aparecer todos los caracteres que no estén incluidos en los corchetes.

\b Fin de palabra. Este símbolo indica que tiene que haber un fin de palabra o retorno de carro.

\B No fin de palabra. El símbolo \B indica cualquiera que no sea un límite de palabra.

\d Cualquier carácter dígito. Este símbolo indica que puede haber cualquier carácter numérico, de 0 a 9.

\D Carácter que no es dígito. Este símbolo indica que puede haber cualquier carácter siempre que no sea numérico.

\f Salto de página. Este símbolo indica que tiene que haber un salto de página.

\n Salto de línea. Este símbolo indica que tiene que haber un salto de línea.

\r Retorno de carro. Este símbolo indica que tiene que haber un retorno de carro.

\s Cualquier espacio en blanco. Este símbolo indica que tiene que haber un carácter individual de espacio en blanco: espacios, tabulaciones, saltos de página o saltos de línea.

\S Carácter que no sea blanco. Este símbolo indica que tiene que haber cualquier carácter individual que no sea un espacio en blanco.

\t Tabulación. Este símbolo indica que tiene que haber cualquier tabulación.

\w Carácter alfanumérico. Este símbolo indica que puede haber cualquier carácter alfanumérico.

\W Carácter que no sea alfanumérico. Este símbolo indica que puede haber cualquier carácter que no sea alfanumérico.

Mediante este lenguaje, se pueden validar elementos de un formulario mediante expresiones regulares. Combinándolas, se pueden abordar infinidad de patrones de validación. Los más clásicos: correo electrónico, teléfono, código postal, NIF, etc.

Un ejemplo de método de validación de correo electrónico con expresiones regulares sería el siguiente:

```
const validaEmail = () => {  
  let valor = document.getElementById("campo").value;  
  return (/^w+([-+.']\w+)*@w+([-.\]\w+)*\.\w+([-.\]\w+)$/).test(valor);  
}
```

Afortunadamente, como ya se vio anteriormente, HTML5 incorpora un tipo de input llamado email que hace ese trabajo sin necesidad de introducir un método que valide un correo electrónico:

```
<label for="email">Correo electrónico:</label>  
<input type="email" id="email" name="email">
```

Otro ejemplo clásico es el del NIF. Para este caso tampoco es estrictamente necesaria la utilización de un método de validación, sino que se puede usar el atributo pattern del elemento input:

```
<label for="nif">NIF:</label>  
<input type="text" id="nif" name="nif" pattern="^\d{8}[A-Za-z]$" title="El DNI debe tener 8 números y 1 letra." />
```

Otro ejemplo: validación de un número de teléfono:

```
<label for="telefono">Número de teléfono:</label>  
<input type="text" id="tel" name="tel" pattern="^\d{9}$" title="El teléfono debe constar de 9 números exclusivamente." />
```

Pero el uso de expresiones regulares no solo se limita a la validación de datos. Otra de las aplicaciones en las que se puede ver la potencia de este lenguaje es el filtrado de datos. Para ver cómo se aplica el filtrado, se va a desarrollar un ejemplo en React de filtrado de usuarios por nombre. Se creará un componente Filtrado (en **Filtrado.jsx**) con un formulario conteniendo un cuadro de texto de búsqueda por nombre. Los resultados aparecerán en una lista.

El código del componente es el siguiente:

```
import { useState } from "react";
const Filtrado = () => {
  const [busqueda, setBusqueda] = useState("");
  const usuarios = [
    { id: 1, nombre: "Cassandra Smith" },
    { id: 2, nombre: "Ryan Curtis" },
    { id: 3, nombre: "Dean Walker" },
  ];
  // Se crea una expresión regular desde la búsqueda
  const regex = new RegExp(busqueda, "i");
  // Se filtra la lista de usuarios
  const usuariosFiltrados = usuarios.filter((usuario) =>
    regex.test(usuario.nombre)
  );
  return (
    <>
      <input type="text" placeholder="Buscar usuario..." value={busqueda}
        onChange={(e) => setBusqueda(e.target.value)} />
      <ul>
        {usuariosFiltrados.map((usuario) => (
          <li key={usuario.id}>{usuario.nombre}</li>
        ))}
      </ul>
    </>
  );
};
export default Filtrado;
```

Aquí se usa la clase **RegExp** proporcionando como parámetros el patrón de búsqueda, el cual representa en sí mismo la expresión regular mientras que el segundo parámetro ("i") indica que no tenga en cuenta mayúsculas ni minúsculas.

Por último, otra de las operaciones que puede realizarse utilizando expresiones regulares es el reemplazo de cadenas. Este tipo de datos en JavaScript tiene un método **replace** el cual se puede usar para sustituir parte de la cadena con otra. Por ejemplo:

```
console.log("papa".replace("p", "m")); // Se obtiene mapa porque solo reemplaza la primera ocurrencia
```

Lo importante es que el primer argumento puede ser una expresión regular. Además, si se quiere que se sustituyan todas las ocurrencias, no solo la primera, hay que añadir una opción **g** a la expresión regular:

```
console.log("Borobudur".replace(/[ou]/, "a")); // Barobudur
console.log("Borobudur".replace(/[ou]/g, "a")); // Barabadar
```

Sobre el uso de expresiones regulares se puede encontrar más información en:

https://developer.mozilla.org/es/docs/Web/JavaScript/Guide/Regular_expressions

7.8 PRUEBA Y DOCUMENTACIÓN DEL CÓDIGO

En anteriores unidades, se han tratado pruebas como las unitarias o las e2e. Continuando con esa línea, en esta unidad se verá cómo diseñar una prueba de integración para una aplicación React. Conviene recordar que son pruebas que aseguran que los componentes interactúan correctamente. Para el ejemplo, se utilizará un formulario similar a los ya vistos:

```
import { useState } from "react";
const Formulario = () => {
  const [nombre, setNombre] = useState("");
  const cambioEntrada = (event) => {
    setNombre(event.target.value);
  };
  const envioForm = (event) => {
    event.preventDefault();
    alert(`Se ha enviado el formulario con el nombre: ${nombre}`);
  };
  return (
    <form onSubmit={envioForm}>
      <label>
        Nombre:
        <input type="text" value={nombre} onChange={cambioEntrada} />
      </label>
      <button type="submit">Enviar datos</button>
    </form>
  );
};
export default Formulario;
```

Con anterioridad se vio que un archivo de pruebas debe tener el mismo nombre del componente anexionando el sufijo test, es decir, en este caso, **Formulario.test.jsx**

En la prueba, en primer lugar, se importan los objetos necesarios para realizarla, tanto del propio núcleo de React como de la biblioteca de pruebas, así como el componente a probar. Al igual que con las unitarias, una prueba se define mediante un nombre y el código que va a ejecutar. Dentro de ella, se renderiza el componente y se obtienen los elementos de interacción del DOM, en este caso, el cuadro de texto y el botón. A continuación, se crea un **mock** que va a observar si se produce un **alert** en el componente. Un **mock** no es ni más ni menos que un objeto que simula un comportamiento de un sistema externo. Después se ejecuta un evento de usuario que escribe un texto y se dispara otro que simula el clic del botón. Esto se hace de forma asíncrona, cuyo mecanismo se estudiará en la próxima unidad.

Para saber si la prueba es correcta, se usa **expect** con el **mock** y un método **toHaveBeenCalledWith** que verifica si la salida del **alert** es la esperada. Al terminar la prueba, se lanza un **restore** sobre él.

La importación de bibliotecas se hace como se vio anteriormente para las unitarias:

```
pnpm add -D vitest @testing-library/react @testing-library/jest-dom jsdom
```

El código de la prueba **Formulario.test.jsx** es el siguiente:

```
import { render, screen, fireEvent } from "@testing-library/react";
import { vi } from "vitest";
import userEvent from "@testing-library/user-event";
import Formulario from "../Formulario";
test("El formulario se envía correctamente con el valor del input", async () => {
  render(<Formulario />); // Se renderiza el componente Formulario
  // Se obtienen los elementos del DOM
  const txtNombre = screen.getByLabelText("Nombre:");
  const btnEnviar = screen.getByText("Enviar datos");
  // Se espía alert para poder capturar su llamada
  const mockAlert = vi.spyOn(window, "alert").mockImplementation(() => {});
  await userEvent.type(txtNombre, "Juan Pérez"); // Se escribe en el cuadro de texto
  fireEvent.click(btnEnviar); // Se simula el clic del botón
  // Ahora se verifica que se haya llamado a alert con el valor correcto
  expect(mockAlert).toHaveBeenCalledWith(
    `Se ha enviado el formulario con el nombre: Juan Pérez`
  );
  // Se restaura la función alert original
  mockAlert.mockRestore();
});
```

Para probar si es correcta, al igual que en otro tipo de prueba, únicamente hay que lanzar **pnpm test** el cual debería dar una salida similar a la de la imagen:

```
PASS src/components/Formulario.test.js
  ✓ El formulario se envía correctamente con el valor del input (142 ms)

Test Suites: 1 passed, 1 total
Tests:       1 passed, 1 total
Snapshots:   0 total
Time:        2.185 s
Ran all test suites.
```

5. Prueba de integración pasada correctamente

En cuanto a documentación, a continuación, se verá un ejemplo de uso de Storybook, una herramienta para crear documentación interactiva de componentes que ya se mencionó en anteriores unidades. Para usar Storybook, el primer paso es integrarla en el proyecto en el que se esté trabajando. Esto se hace mediante la siguiente orden:

```
npx storybook init
```

Básicamente, lo que hará una vez que se termine de instalar es crear un directorio **.storybook** y otro en **src/stories** en el que albergar las historias que se diseñen además de algunas que

incluye a modo de ejemplo. Además, instalará las dependencias que necesite.

Antes de iniciar el servidor local en el que se mostrarán las distintas **stories**, se va a crear una personalizada con el ejemplo de formulario anterior, aunque hay que realizarle una serie de modificaciones.

La primera es tener en cuenta que hay que añadirle un par de **props** para que pueda ser compatible con las stories a diseñar. El código modificado sería el siguiente:

```
import { useState } from "react";
const Formulario = ({ initialState = "", onChange }) => {
  const [nombre, setNombre] = useState(initialState);
  const cambioEntrada = (event) => {
    const valor = event.target.value;
    setNombre(valor);
    if (onChange) {
      onChange(valor); // Notifica al padre si hay un manejador de cambios
    }
  };
  const envioForm = (event) => {
    event.preventDefault();
    alert(`Se ha enviado el formulario con el nombre: ${nombre}`);
  };
  return (
    <form onSubmit={envioForm}>
      <label>
        Nombre:
        <input type="text" value={nombre} onChange={cambioEntrada} />
      </label>
      <button type="submit">Enviar datos</button>
    </form>
  );
};
export default Formulario;
```

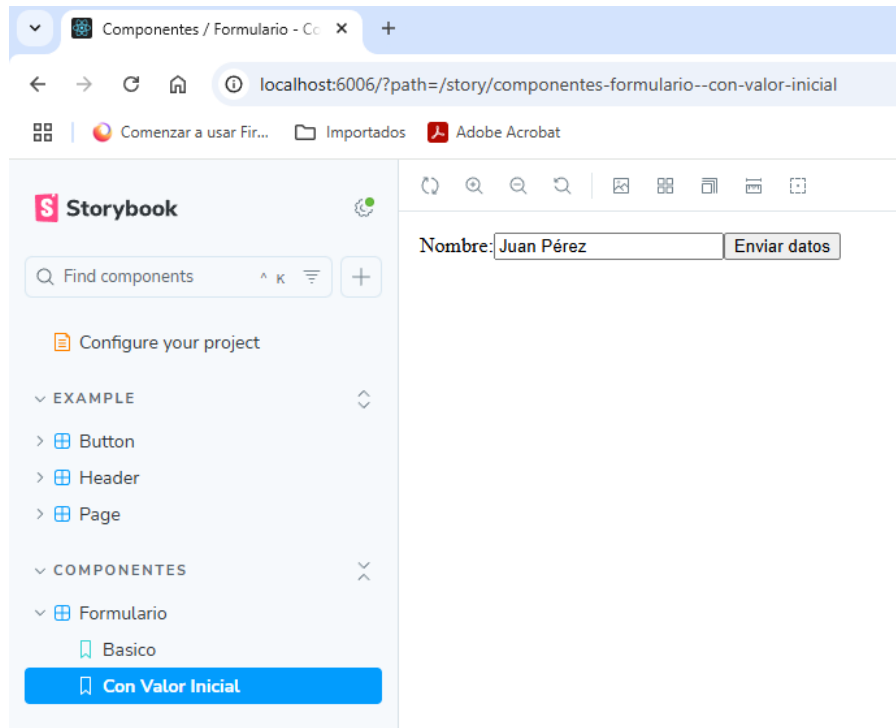
Ahora es cuando se procede a crear la story (como src/stories/Formulario.stories.jsx):

```
import Formulario from "../components/Formulario";
export default {
  title: "Componentes/Formulario", // Título de la sección en Storybook
  component: Formulario, // El componente que se está documentando
};
// Historia básica
export const Basico = () => <Formulario />;
// Historia con valor inicial
export const ConValorInicial = () => <Formulario initialState="Juan Pérez" />;
```

Ahora se ejecuta al fin el storybook mediante la orden:

```
pnpm run storybook
```

En este momento se abrirá un servidor local con un interfaz web en el que pueden verse las distintas stories e incluso probar el componente de forma aislada:



6. Componente en Storybook

ÍNDICE DE FIGURAS

1. Ejemplo de formulario simple	14
2. Comparación entre botón sin estilo aplicado y otro con texto rojo y borde inferior azul	18
3. Cuadros de texto sin y con padding.	19
4. Formulario básico organizado	19
5. Prueba de integración pasada correctamente	29
6. Componente en Storybook.....	31

BIBLIOGRAFÍA - WEBGRAFÍA

Vara Mesa, J.M. et al (2012). *Desarrollo Web en Entorno Cliente*. Editorial Ra-Ma.

Altadill Izura, P.X. (2023). *React Práctico*. Editorial Anaya

Haverbeke, M. (2024). *JavaScript elocuente*. Editorial Anaya.

Barklund, M. (2024). *React in depth*. Editorial O'Reilly.

Maza, A. (2024) *React: Ampliando conceptos*.

<https://openwebinars.net/academia/portada/react-intermedio/> OpenWebinars

Patil, R. (2024) *How to Build Forms in React*. <https://medium.com/@rashmipatil24/forms-in-react-d9029eeca5b5>

Casero, A. (2024) *¿Qué es FormData en JavaScript?* <https://keepcoding.io/blog/formdata-en-javascript/>

Angular Minds (2024). *React Synthetic Events for Efficient Event Handling*-
<https://www.angularminds.com/blog/react-synthetic-events-for-efficient-event-handling>